UNIVERSITY OF RWANDA

CST

COMPUTER ENGINEERING

LEVEL IV

GAKO CAMPUS

REAL TIME SYSTEMS

Submission date: [27/10/2026]

REAL TIME SYSTEM MINI PROJECT 2

Unit 2: Scheduling, Priority Inversion and Multiprocessor Partitioning

---

GROUP 4_MEMBERS:

ABDUL SIBO **223026681**

JUSTINE IRAFASHA **223026693**

FILS NIYONZIMA **223026876**

Table of Contents

Mini Project 2.1: Simulating RM, EDF and Priority Inheritance

Utilization tests (Steps 2-3)
RM and EDF schedules (Steps 4-6)
Response Time Analysis (Step 7)
Priority inversion and PIP (Steps 8-10)
Answers to Lab Tasks: Project 2.1
Conclusion: Project 2.1

Mini Project 2.2: RM versus EDF Schedulability Comparison Tool

Results for six task sets
Written report: Project 2.2
Reflection: when the bound fails but RM still works
Conclusion: Project 2.2

Mini Project 2.3: Partitioned Multiprocessor Scheduling Simulator

Test sets
Heuristic comparison
Adversarial set
Per-core schedule of a partitioned configuration
Written analysis: Project 2.3
Conclusion: Project 2.3

Appendix: Source Code

A. Mini Project 2.1
B. Mini Project 2.2
C. Mini Project 2.3

Mini Project 2.1: Simulating RM, EDF and Priority Inheritance

Setup. A pure-Python simulator (rts_sim.py) was built around a Task record (name, C, T, D with D = T by default, and U = C/T), a tick-by-tick scheduler that, at every unit of time, releases the jobs due and runs the ready job chosen by the policy (shortest period for RM, shortest relative deadline for DM, earliest absolute deadline for EDF, least laxity for LLF), Response Time Analysis, and a separate small simulator for the shared-resource scenario with and without the Priority Inheritance Protocol. Charts use matplotlib's broken_barh. Ties are broken in favour of the job already running, which avoids needless preemptions.

Utilization tests (Steps 2-3)

Task set	Total U	RM bound n(2^(1/n)-1)	RM sufficient test	EDF U ≤ 1
t1(1,4), t2(2,6), t3(2,12)	0.750	0.780	PASS	PASS
+ t4(1,8)	0.875	0.757	FAIL (inconclusive)	PASS

Both worked examples reproduce the manual exactly: U = 0.750 against a bound of 0.780, then U = 0.875 against 0.757, so the RM test cannot guarantee the four-task set while EDF accepts it.

RM and EDF schedules (Steps 4-6)

Both policies were simulated over the 24-tick hyperperiod of the four-task set. Neither missed a deadline. The schedules agree on t1 but differ at 6 of the 24 ticks (ticks 6, 8, 9, 10, 17, 21) as the deadlines of t2, t3 and t4 interleave: for example EDF runs t3 for ticks 5-6 straight through because its deadline at 12 is nearer than the next t2 deadline, while RM always gives t2 priority.

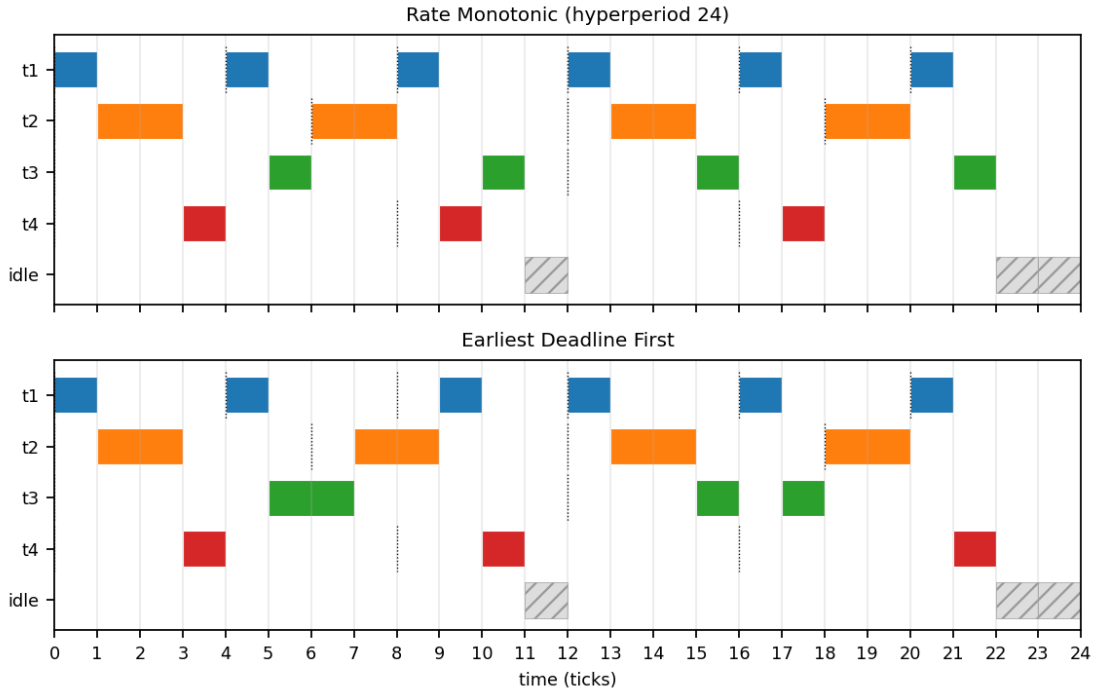


Figure 1. RM (top) and EDF (bottom) schedules of the four-task set over its 24-tick hyperperiod; dotted lines mark job releases and hatched cells are idle time.

Response Time Analysis (Step 7)

Task (priority order)	C	T = D	Response time R	Iterations	R ≤ D
t1	1	4	1	1	yes
t2	2	6	3	2	yes
t4	1	8	4	2	yes
t3	2	12	11	5	yes

RTA gives a definitive answer where the utilization bound was inconclusive. For t4, R = 4 ≤ 8 after two iterations (1 → 4 → 4). The tightest task is t3 with R = 11 against D = 12, so RM does schedule the four-task set even though the Liu & Layland bound failed.

Priority inversion and PIP (Steps 8-10)

L (priority 1) acquires the shared resource at t = 0 and needs 6 ticks, all inside the critical section; H (priority 3) arrives at t = 1 and needs the resource for 2 ticks; M (priority 2) arrives at t = 2, needs 4 ticks and does not use the resource.

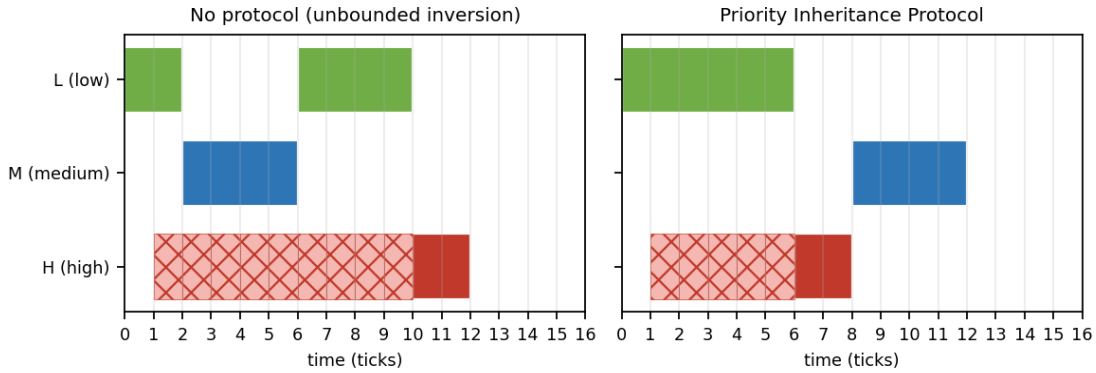


Figure 2. Left: no protocol, M preempts L and H is blocked from t = 1 to t = 10. Right: with PIP, L inherits H's priority and H is blocked only from t = 1 to t = 6. Hatched red = H blocked.

Run	H blocked (ticks)	H finishes	M finishes	L finishes	Bound (one critical section of L)
No protocol	9	12	6	10	not bounded
PIP	5	8	12	6	6 ticks

Why the PIP blocking matches the bound (Section 2.7.1).

Without a protocol, H arrives at t = 1 and needs the resource that L has held since t = 0, so H blocks. L has the lowest priority, so when M arrives at t = 2 it preempts L, and L cannot finish its critical section. H therefore waits for the remaining 5 ticks of L's critical section and for all 4 ticks of M, a total of 9 ticks, although M never touches the resource and ranks below H. When a second medium-priority task was added, the wait rose to 12 ticks, so without PIP the blocking has no bound other than the amount of medium-priority work that happens to arrive. With PIP, when H blocks at t = 1 the kernel raises L to H's priority, so M can no longer preempt L. L completes its critical section at t = 6, releases the resource and returns to its base priority, and H runs at once. H's blocking is 5 ticks. Section 2.7.1 bounds blocking by one critical section of each lower-priority task that shares the resource; here that is a single critical section of L, 6 ticks long. The measured 5 is one tick below the bound only because L had already executed one tick of its section before H arrived; if H had arrived at the instant L took the lock, the blocking would equal the full

6 ticks. The second medium task changed nothing under PIP (still 5 ticks), which matches the bound: it depends on the lower-priority tasks that hold the resource, not on how many unrelated medium tasks arrive.

Answers to Lab Tasks: Project 2.1

Lab Task 1: Deadline Monotonic. For an implicit-deadline set (a: C=2, T=6; b: C=2, T=8) the RM and DM timelines are identical (identical = true), because ordering by period and ordering by deadline are the same ordering. When task b is given D = 3, the timelines differ (identical = false): RM, which still ranks by period, runs a first and b finishes at tick 4, missing its deadline of 3 (1 miss), whereas DM ranks b first and meets every deadline (0 misses). The schedule therefore changes only when some D differs from T.

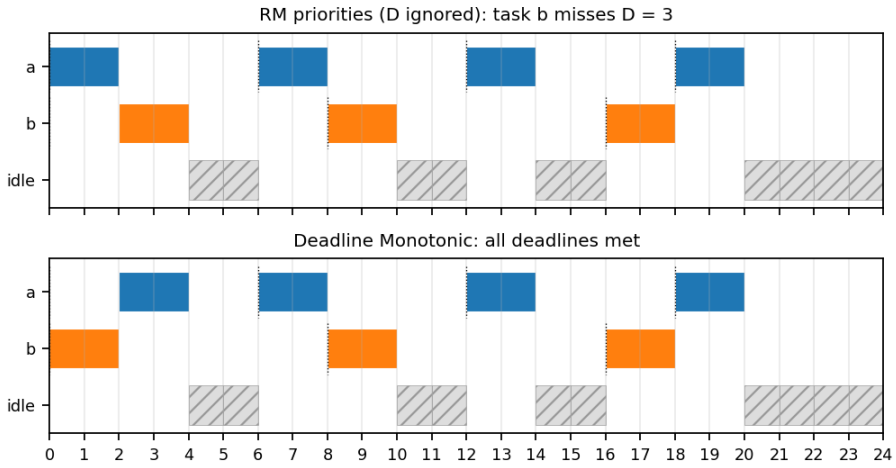


Figure 3. RM versus DM priorities for a constrained-deadline set (b has D = 3).

Lab Task 2: bound sweep. rm_bound(n) for n = 1 to 10 is 1.000, 0.828, 0.780, 0.757, 0.743, 0.735, 0.729, 0.724, 0.721, 0.718. It starts at 1.0 (one task can use the whole processor), falls quickly (0.828 at n = 2, 0.780 at n = 3) and then flattens, still 0.025 above ln 2 = 0.693 at n = 10. The EDF bound stays constant at 1.0, so the guaranteed gap between EDF and RM widens from 0 to about 30 percentage points.

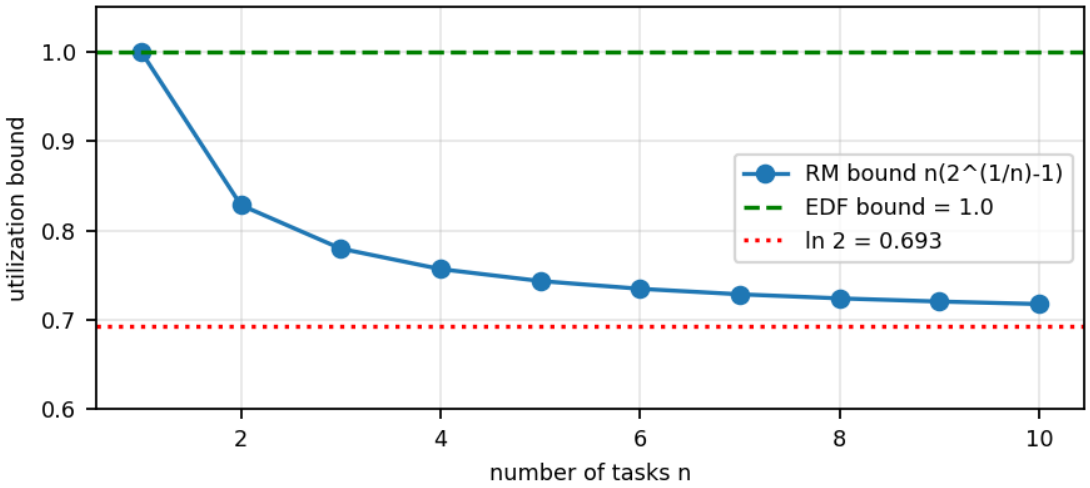


Figure 4. RM utilization bound versus number of tasks, against EDF's constant bound of 1.0 and the ln 2 limit.

Lab Task 3: Least Laxity First. Over the hyperperiod of the four-task set, RM made 15 context switches, EDF 14 and LLF 15, so LLF showed no thrashing on that set. LLF thrashes when jobs have equal laxity, because the running job's laxity stays constant while the waiting job's laxity falls, so the two alternate every tick. A

set of three tasks with T = 8 and C = 3, 3, 2 shows this: over one hyperperiod RM made 2 switches, EDF 2 and LLF 5. LLF is therefore an optimal policy in theory but needs a tie rule or hysteresis to be usable.

Lab Task 4: second medium task. A second medium-priority task M2 (arrives at t = 4, needs 3 ticks) increased H's blocking without protection from 9 to 12 ticks (H finishes at 15), and left it unchanged with PIP at 5 ticks (H finishes at 8). The unprotected inversion grows with every medium task, while the PIP bound does not.

Conclusion: Project 2.1

The simulator reproduced the numbers of Sections 2.2 and 2.3 exactly, showed that exact analysis (RTA) can accept a set the utilization bound rejects, and reproduced both unbounded priority inversion and its bounded PIP fix, in the spirit of the Mars Pathfinder case.

Mini Project 2.2: RM versus EDF Schedulability Comparison Tool

Setup. sched_tool.py reads a task set from a Python list or a CSV file (columns name, C, T and optional D). It validates the input, computes per-task and total utilization, evaluates the Liu & Layland bound, falls back to Response Time Analysis when the bound fails (or when any D < T, where the bound does not apply and Deadline Monotonic ordering is used), evaluates EDF's U ≤ 1 condition, confirms both verdicts by simulation over the hyperperiod (capped at 500 ticks), and draws a two-panel RM/EDF chart with idle time hatched. Invalid input (zero or negative values, D > T, C > D) returns a message, and U > 1 is reported as infeasible without a chart. A CSV test and a bad-input test both ran correctly.

Results for six task sets

Task set	n	U	LL bound	Bound test	RTA (exact)	EDF U ≤ 1
S1 Section 2.2.2 (3 tasks)	3	0.750	0.780	PASS	not needed	PASS
S2 Section 2.3.2 (4 tasks)	4	0.875	0.757	FAIL	all R ≤ D	PASS
S3 overloaded (U > 1)	3	1.517	0.780	FAIL	infeasible	FAIL
S4 constrained deadline	3	0.667	0.780	n/a (D < T)	all R ≤ D	PASS
S5 eight tasks, U = 0.70	8	0.700	0.724	PASS	not needed	PASS
S6 eight harmonic tasks, U = 0.80	8	0.800	0.724	FAIL	all R ≤ D	PASS

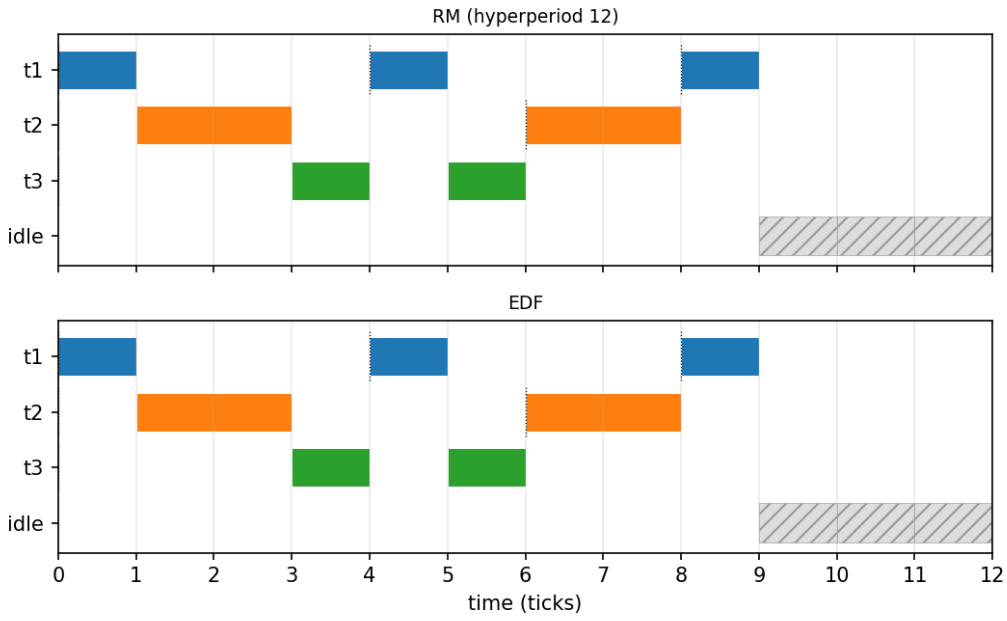


Figure 5. S1 (Section 2.2.2): RM above, EDF below; the bound test passes and both schedules meet every deadline.

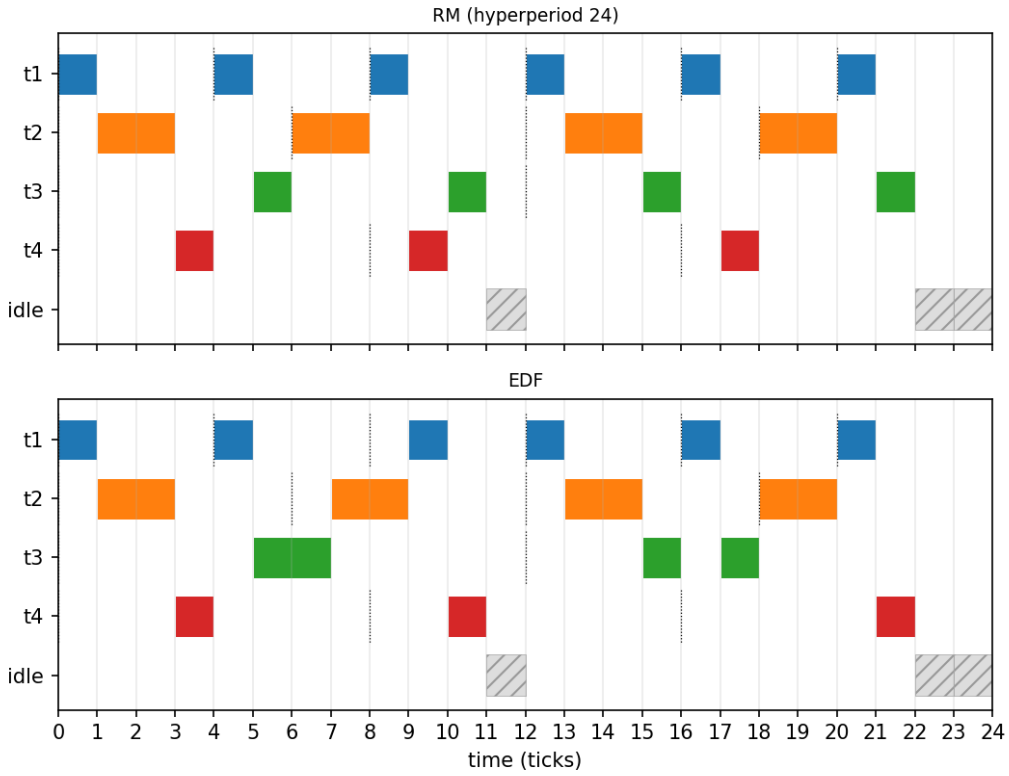


Figure 6. S2 (Section 2.3.2): the bound fails, RTA proves RM schedulable, and the two schedules diverge among t2, t3 and t4.

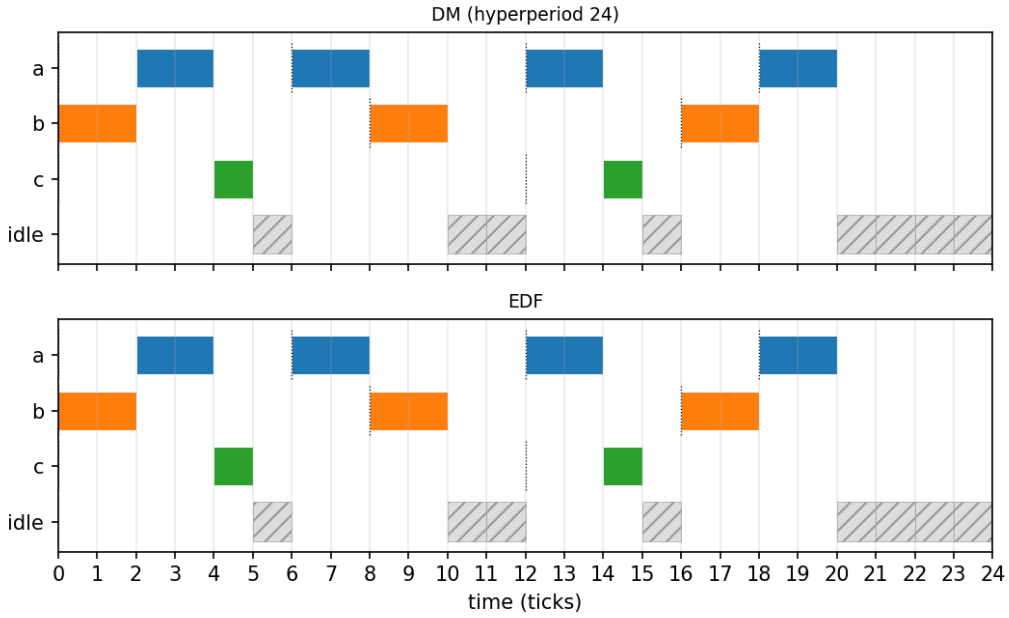


Figure 7. S4: constrained deadline (b has D = 3), Deadline Monotonic order used for the fixed-priority panel.

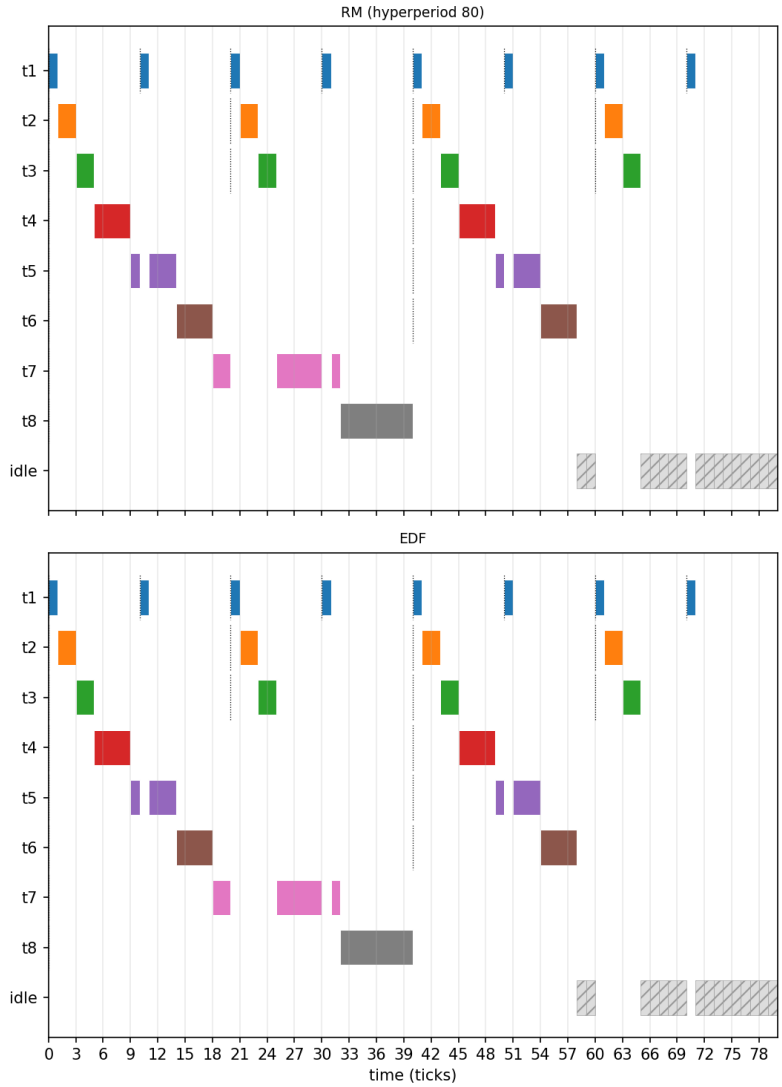


Figure 8. S6: eight harmonic tasks, U = 0.80, bound 0.724 fails but RTA shows RM still meets all deadlines.

Written report: Project 2.2

Six task sets were analysed with the tool; the table above summarises them. Set S1 is the three-task example of Section 2.2.2. Its utilization is 0.750 and the Liu & Layland bound for three tasks is 0.780, so the sufficient test passes and RM is guaranteed; EDF also passes since U is below 1. Set S2 adds the fourth task of Section 2.3.2. Utilization rises to 0.875, above the four-task bound of 0.757, so the sufficient test is inconclusive and the tool falls back to Response Time Analysis. The RTA values (R = 1, 3, 4 and 11 for tasks t1, t2, t4 and t3 in priority order) are all below their deadlines, so RM does schedule this set, and the simulation over the 24-tick hyperperiod confirms zero misses under both RM and EDF. The contrast the manual draws is therefore subtler than "RM fails, EDF succeeds": the bound cannot guarantee RM here, but exact analysis shows RM still works, with t3 finishing at 11 against a deadline of 12.

Set S3 is deliberately overloaded, with U = 1.517. The tool reports it as infeasible, states that no uniprocessor scheduler can meet all deadlines, skips the simulation and chart, and does not crash. Set S4 has a constrained deadline (task b has C = 2, T = 8 and D = 3). The tool switches to Deadline Monotonic ordering (b, a, c), recognises that the Liu & Layland bound assumes D = T and does not apply, and runs RTA directly: R = 2, 4 and 5 against deadlines 3, 6 and 12. EDF's U ≤ 1 condition is only a necessary condition

when D < T, and the tool says so; the simulation shows no misses under EDF either. Under plain RM priorities, task b would miss its deadline of 3, which is why the DM ordering matters.

Sets S5 and S6 each contain eight tasks with periods 10 to 80, so the bound is near its asymptotic value of ln 2: n(2^(1/n) − 1) = 0.724 for n = 8, against 0.780 for n = 3. S5 has U = 0.70, just under the bound, so the sufficient test passes immediately and no RTA is needed. S6 is harmonic with U = 0.80, above the bound, so the test fails; RTA nevertheless shows every task meets its deadline, with the largest response time being 40 against 80 for the slowest task. EDF passes both sets.

Across the six sets the pattern is consistent. The bound is quick but pessimistic, and it shrinks toward 0.693 as n grows, so it rejects more sets the larger they are. RTA is exact for fixed priorities and resolves every inconclusive case here. EDF's U ≤ 1 test is exact for implicit deadlines and accepted every feasible set, which is the 100% versus roughly 69% to 78% utilization gap of Section 2.3.

Reflection: when the bound fails but RM still works

Set S2 (and, more strikingly, the harmonic set S6) shows why exact tests matter. For S2, utilization is 0.875 and the four-task bound is 0.757, so the Liu & Layland test cannot guarantee RM. A designer relying on the bound alone would have to reject the design, or move to EDF, or add a faster processor. Response Time Analysis tells a different story: t3, the lowest-priority task, has a response time of 11 against a deadline of 12, and every other task finishes earlier, so RM does schedule the set and the simulation over the hyperperiod confirms it. The bound is only sufficient, never necessary. It must hold for the worst possible period ratios, but this set has period ratios (4, 6, 8, 12) that are far from that worst case. S6 makes the point more strongly: with U = 0.80 against a bound of 0.724, the harmonic periods let RM schedule the set with a worst response time of half the deadline. The cost of the exact test is a short iterative calculation per task, and the benefit is that a schedulable design is not rejected. The margin is thin for t3 in S2, which RTA also reveals and the bound cannot.

Conclusion: Project 2.2

The tool combines three layers: a fast sufficient test, an exact fixed-priority test used when the first is inconclusive, and a simulation that cross-checks both. Each of the six sets produced a clear verdict, and the infeasible and constrained-deadline cases were handled without error.

Mini Project 2.3: Partitioned Multiprocessor Scheduling Simulator

Setup. partition.py implements four bin-packing heuristics behind one signature (tasks, m) that returns a core-to-task assignment plus the list of unplaced tasks. The admission predicate is the Liu & Layland bound recomputed for the number of tasks the core would then hold: a task of utilization u is admissible on a core with load L and n tasks if L + u ≤ rm_bound(n + 1). FF scans cores in order for the first admissible one; FFD first sorts tasks by decreasing u and then applies FF; BFD sorts decreasing and chooses the admissible core with the least remaining capacity afterwards; WF (unsorted) chooses the admissible core with the most remaining capacity. All runs use m = 4 cores.

Test sets

Set	Tasks	Total U	% of m = 4	Description
S1	16	2.600	65.0	Balanced: 16 small tasks of u = 0.15 and 0.175
S2	13	2.800	70.0	Skewed: 10 tasks of u = 0.1 listed first, then 3 of u = 0.6
S3	16	2.880	72.0	Tight: 16 tasks of u = 0.18, exactly four per core
S4	5	2.550	63.7	Adversarial: 5 tasks of u = 0.51
S5	10	2.975	74.4	Order-sensitive mixed set (found by random search)

Heuristic comparison

Set	FF	FFD	BFD	WF
S1	4 cores, 65.0%	4 cores, 65.0%	4 cores, 65.0%	4 cores, 65.0%
S2	FAIL (1 unplaced), 55.0%	4 cores, 70.0%	4 cores, 70.0%	FAIL (3 unplaced), 25.0%
S3	4 cores, 72.0%	4 cores, 72.0%	4 cores, 72.0%	4 cores, 72.0%
S4	FAIL (1 unplaced), 51.0%	FAIL (1 unplaced), 51.0%	FAIL (1 unplaced), 51.0%	FAIL (1 unplaced), 51.0%
S5	FAIL (1 unplaced), 65.6%	4 cores, 74.4%	4 cores, 74.4%	FAIL (1 unplaced), 65.6%

Each cell shows cores used and the share of the 4-core capacity actually placed, or FAIL with the number of tasks that could not be placed. The lower bound ceil(U) on the number of cores is 3 for S1, S2, S3 and S5 and 3 for S4, so any set that FFD fits on four cores is within one core of that bound.

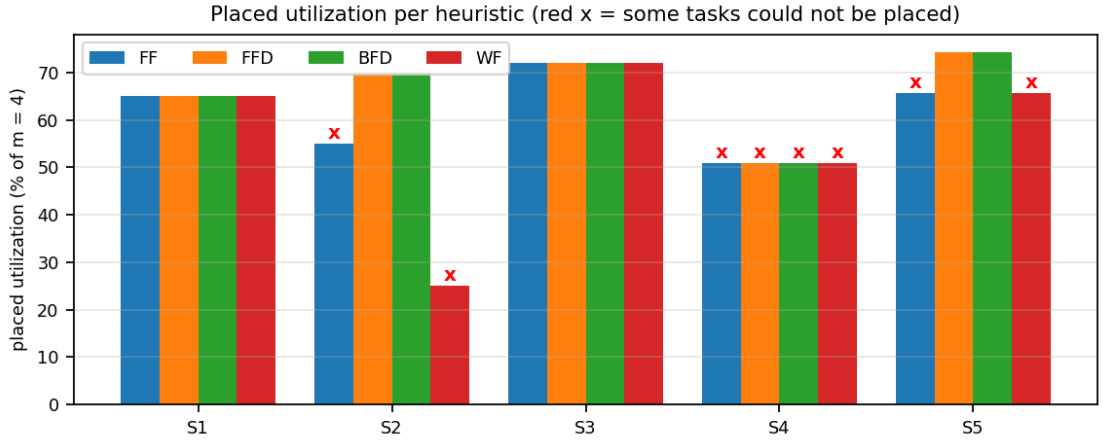


Figure 9. Placed utilization as a percentage of the 4-core capacity for each heuristic and set (red x = at least one task unplaced).

Adversarial set

S4 holds five tasks of u = 0.51, a total of 2.55, only 64% of capacity. Every heuristic placed four tasks, one per core, and failed on the fifth.

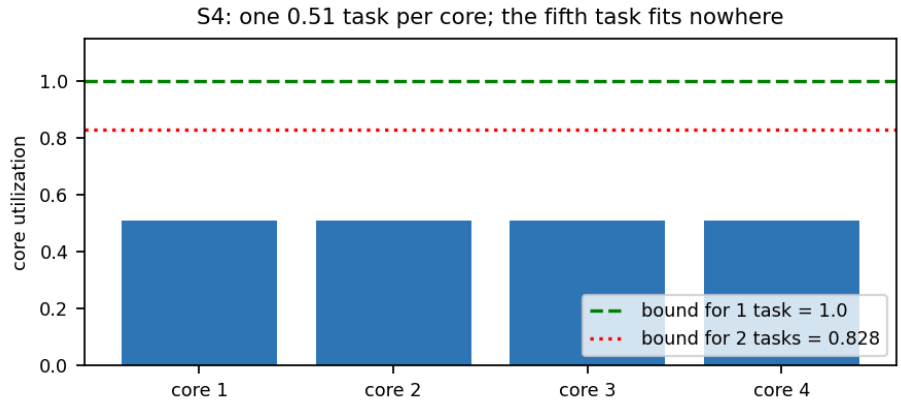


Figure 10. S4: each core holds one 0.51 task, well under the two-task bound of 0.828 for any second 0.51 task.

Per-core schedule of a partitioned configuration

The FFD partition of S5 was simulated core by core with the RM simulator from Project 2.1 over the 40-tick hyperperiod. Every core met every deadline (misses per core: 0, 0, 0, 0), confirming that the admission test makes each core independently schedulable.

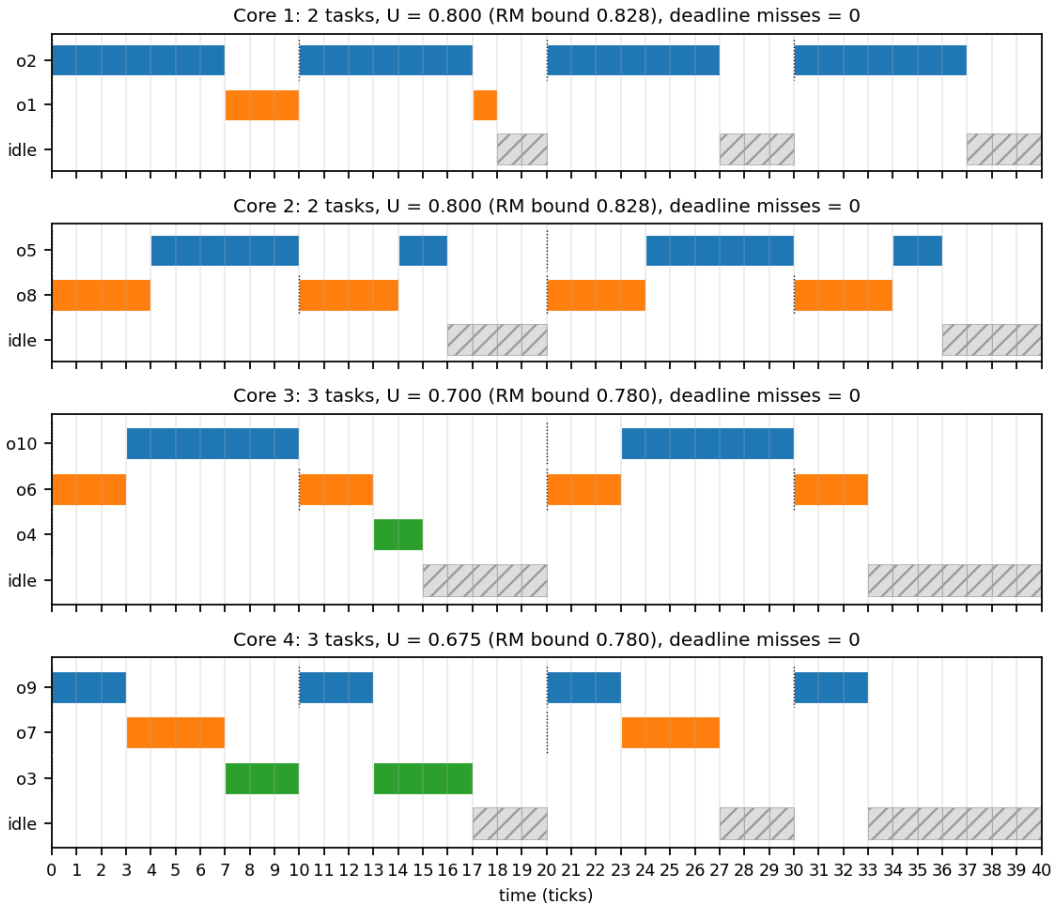


Figure 11. FFD partition of S5: one panel per core, each simulated independently under RM; no deadline misses.

Written analysis: Project 2.3

Section 2.8.1 reduces partitioned scheduling to m independent uniprocessor problems plus a bin-packing problem, and bin packing is hard, so practical systems use heuristics (Section 2.8.3). Here each core's admission test is the Liu & Layland bound recomputed for the new task count, so a core holding n tasks accepts another only while load + u ≤ rm_bound(n + 1). Five sets were packed onto m = 4 cores with FF, FFD, BFD and WF.

The results support FFD over plain FF and WF. On the balanced set S1 and the tight set S3, where all tasks are similar in size, all four heuristics succeed and give identical total utilization, so ordering does not matter. On the skewed set S2 the picture changes. With the ten small tasks listed before the three tasks of u = 0.6, FF fills the first core with seven small tasks and spills the rest onto a second; the third big task then finds no core that can take it (0.3 + 0.6 exceeds the four-task bound), and FF leaves one task unplaced. WF is worse still: it spreads the small tasks evenly, so every core already carries at least 0.2 and none of the three big tasks fits anywhere, leaving 25% of m placed. FFD and BFD, which place the big tasks first and then fit small tasks around them, packed all 13 tasks at 70% of m. On the order-sensitive set S5, FF and WF failed to place the tenth task, while FFD and BFD placed everything and reached 74.4% of m. Sorting by decreasing utilization places the awkward large items while the cores are still empty, which is the same reason first-fit decreasing is a better bin-packing heuristic in general.

BFD and FFD gave the same success pattern here; they differ in which core they choose, not in whether they succeed on these sets. For S5, per-core utilization under FFD was 0.80, 0.80, 0.70 and 0.675, and the per-core RM simulation over the 40-tick hyperperiod showed zero deadline misses on every core, confirming that the admission test did its job and that each core can be analysed on its own.

The adversarial set S4 contains five tasks of u = 0.51, total U = 2.55, only 64% of the 4-core capacity. Two such tasks cannot share a core, because 1.02 exceeds the two-task bound of 0.828, so each core takes one task and the fifth cannot be placed under any heuristic. This is the bin-packing limitation of Section 2.8.1: total utilization well below m does not make a set partitionable, because no individual core can accept a second task whose size is just over half a core. A simple lower bound of ceil(U) = 3 cores would suggest it fits, but the real requirement is five cores. The lesson is that partitioning feasibility depends on the distribution of utilizations, not just their sum, and that large tasks must be placed first.

Conclusion: Project 2.3

The simulator shows that partitioned scheduling succeeds or fails on the shape of the utilization distribution, not on the total. Sorting by decreasing utilization (FFD, BFD) was never worse than FF or WF in these experiments and was decisive on the skewed and order-sensitive sets, while the adversarial set defeated all four heuristics despite using only 64% of capacity.

Appendix: Source Code

Complete source of the scripts used to produce the results in this report.

A. Mini Project 2.1

rts_sim.py

```
"""Mini Project 2.1: RM / DM / EDF / LLF tick simulator, RTA, and priority inversion with PIP (Unit 2)."""
import math
from dataclasses import dataclass
from functools import reduce

@dataclass
class Task:
    name: str
    C: int
    T: int
    D: int = None                       # relative deadline, default D = T (implicit)
    def __post_init__(self):
        if self.D is None: self.D = self.T
    @property
    def U(self): return self.C / self.T

def total_utilization(tasks): return sum(t.U for t in tasks)
def rm_bound(n): return n * (2 ** (1 / n) - 1)               # Liu & Layland, Section 2.2.1
def hyperperiod(tasks): return reduce(lambda a, b: a * b // math.gcd(a, b), (t.T for t in tasks))

def priority_order(tasks, policy="RM"):
    """Static priority order, highest first (RM by period, DM by deadline)."""
    key = (lambda t: t.T) if policy == "RM" else (lambda t: t.D)
    return sorted(tasks, key=key)                            # stable: ties keep input order

def rta(ordered, i):
    """Response Time Analysis (Section 2.5.2) for ordered[i]. Returns (R, iterations, meets_deadline)."""
    C, R, it = ordered[i].C, ordered[i].C, 0
    while True:
        it += 1
        new = C + sum(math.ceil(R / h.T) * h.C for h in ordered[:i])
        if new == R: return R, it, R <= ordered[i].D
        if new > ordered[i].D: return new, it, False
        R = new

def simulate(tasks, sim_time, policy="RM"):
    """Tick-by-tick simulation, unit time steps. policy in RM, DM, EDF, LLF.
    Returns dict(timeline=[task name or None per tick], misses=[...], switches=int)."""
    jobs, timeline, running = [], [], None
    for t in range(sim_time):
        for i, k in enumerate(tasks):
            if t % k.T == 0:
                jobs.append(dict(task=k, idx=i, rem=k.C, d=t + k.D, rel=t, id=(k.name, t // k.T), fin=None))
        ready = [j for j in jobs if j["rem"] > 0]
        if not ready:
            timeline.append(None); running = None; continue
        def key(j):
            p = {"RM": j["task"].T, "DM": j["task"].D, "EDF": j["d"], "LLF": j["d"] - t - j["rem"]}[policy]
            return (p, 0 if j is running else 1, j["idx"])    # ties: keep the running job, then input order
        cur = min(ready, key=key)
        cur["rem"] -= 1; running = cur
        timeline.append(cur["task"].name)
        if cur["rem"] == 0: cur["fin"] = t + 1
    return _finish(jobs, timeline, sim_time, tasks, policy)

def _finish(jobs, timeline, sim_time, tasks, policy):
    misses = [(j["id"], j["d"], j["fin"]) for j in jobs if (j["fin"] is None and j["d"] <= sim_time) or
(j["fin"] is not None and j["fin"] > j["d"])]
    return dict(timeline=timeline, misses=misses, switches=count_switches(tasks, sim_time, policy))

def count_switches(tasks, sim_time, policy):
    jobs, running, prev, sw = [], None, None, 0
    for t in range(sim_time):
        for i, k in enumerate(tasks):
            if t % k.T == 0: jobs.append(dict(task=k, idx=i, rem=k.C, d=t + k.D))
        ready = [j for j in jobs if j["rem"] > 0]
        if not ready: running = None; prev = None; continue
        def key(j):
```

```
            p = {"RM": j["task"].T, "DM": j["task"].D, "EDF": j["d"], "LLF": j["d"] - t - j["rem"]}[policy]
            return (p, 0 if j is running else 1, j["idx"])
        cur = min(ready, key=key); cur["rem"] -= 1; running = cur
        if prev is not None and cur is not prev: sw += 1
        prev = cur
    return sw

def simulate_rm(tasks, sim_time): return simulate(tasks, sim_time, "RM")
def simulate_edf(tasks, sim_time): return simulate(tasks, sim_time, "EDF")

# ---------------- priority inversion / PIP (Sections 2.6, 2.7) ----------------
def simulate_inversion(jobs, sim_time, pip):
    """jobs: dicts(name, prio (bigger = higher), arrive, C, res). A job with res=True needs the single
shared
    resource for its whole execution; it acquires it the first time it runs and releases it on
completion."""
    st = {j["name"]: dict(j, rem=j["C"], fin=None) for j in jobs}
    holder, timeline, blocked_ticks, hold = None, [], {n: 0 for n in st}, []
    for t in range(sim_time):
        ready = [s for s in st.values() if s["arrive"] <= t and s["rem"] > 0]
        eff = {s["name"]: s["prio"] for s in ready}
        blocked = [s for s in ready if s["res"] and holder is not None and holder != s["name"]]
        if pip:
            for b in blocked: eff[holder] = max(eff[holder], eff[b["name"]])     # inherit highest blocked
priority
        runnable = [s for s in ready if s not in blocked]
        if not runnable: timeline.append((None, [])); continue
        cur = max(runnable, key=lambda s: (eff[s["name"]], -s["arrive"]))
        if cur["res"] and holder is None: holder = cur["name"]
        cur["rem"] -= 1
        for s in ready:
            if s is not cur: blocked_ticks[s["name"]] += 1
        timeline.append((cur["name"], [b["name"] for b in blocked]))
        hold.append(holder)
        if cur["rem"] == 0:
            cur["fin"] = t + 1
            if holder == cur["name"]: holder = None
    return dict(timeline=timeline, finish={n: s["fin"] for n, s in st.items()}, waiting=blocked_ticks,
hold=hold)

INV_JOBS = [dict(name="L", prio=1, arrive=0, C=6, res=True),
            dict(name="H", prio=3, arrive=1, C=2, res=True),
            dict(name="M", prio=2, arrive=2, C=4, res=False)]

# ---------------- plotting ----------------
def _segments(timeline, names):
    segs = {n: [] for n in names}
    for t, n in enumerate(timeline):
        if n is None: continue
        if segs[n] and segs[n][-1][0] + segs[n][-1][1] == t: segs[n][-1] = (segs[n][-1][0], segs[n][-1][1] +
1)
        else: segs[n].append((t, 1))
    return segs

def draw_schedule(ax, tasks, res, title, horizon, show_idle=True):
    import matplotlib.pyplot as plt
    cols = plt.get_cmap("tab10")
    names = [t.name for t in tasks]; segs = _segments(res["timeline"], names)
    for i, n in enumerate(names):
        ax.broken_barh(segs[n], (i - .35, .7), facecolors=cols(i), edgecolor="white", linewidth=.5)
    rows = len(names)
    if show_idle:
        idle = [(t, 1) for t, n in enumerate(res["timeline"]) if n is None]
        ax.broken_barh(idle, (rows - .35, .7), facecolors="#dddddd", hatch="///", edgecolor="#999999",
linewidth=.3)
        ax.set_yticks(range(rows + 1), names + ["idle"])
    else: ax.set_yticks(range(rows), names)
    for i, k in enumerate(tasks):                                    # release markers
        for r in range(0, horizon, k.T): ax.plot([r, r], [i - .45, i + .45], color="k", lw=.6, ls=":")
    ax.set_xlim(0, horizon); ax.set_xticks(range(0, horizon + 1, max(1, horizon // 24)))
    ax.invert_yaxis(); ax.set_title(title, fontsize=9); ax.grid(axis="x", alpha=.25)

def draw_inversion(ax, result, title, horizon):
    import matplotlib.pyplot as plt
    order = ["H", "M", "L"]; cols = {"H": "#c0392b", "M": "#2e75b6", "L": "#70ad47"}
    tl = [n for n, _ in result["timeline"]]
    segs = _segments(tl, order)
    for i, n in enumerate(order): ax.broken_barh(segs[n], (i - .35, .7), facecolors=cols[n],
edgecolor="white")
    blk = [(t, 1) for t, (n, b) in enumerate(result["timeline"]) if "H" in b]
```

```
    ax.broken_barh(blk, (-.35, .7), facecolors="#f5b7b1", hatch="xx", edgecolor="#c0392b", linewidth=.3)
    ax.set_yticks(range(3), ["H (high)", "M (medium)", "L (low)"]); ax.set_xlim(0, horizon);
ax.set_xticks(range(horizon + 1))
    ax.invert_yaxis(); ax.set_title(title, fontsize=9); ax.grid(axis="x", alpha=.25); ax.set_xlabel("time
(ticks)")

if __name__ == "__main__":
    A = [Task("t1", 1, 4), Task("t2", 2, 6), Task("t3", 2, 12)]
    print(f"3-task set: U = {total_utilization(A):.3f}, rm_bound(3) = {rm_bound(3):.3f}, RM sufficient test
passes: {total_utilization(A) <= rm_bound(3)}")
    B = A + [Task("t4", 1, 8)]
    print(f"4-task set: U = {total_utilization(B):.3f}, rm_bound(4) = {rm_bound(4):.3f}, RM sufficient test
passes: {total_utilization(B) <= rm_bound(4)}, EDF U<=1: {total_utilization(B) <= 1}")
    o = priority_order(B)
    for i, k in enumerate(o):
        R, it, ok = rta(o, i); print(f"  RTA {k.name}: R = {R} (iterations {it}) <= D = {k.D}: {ok}")
```

exp21.py

```
import json, math, matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from rts_sim import *
plt.rcParams.update({"font.size": 8})
R = {}
A = [Task("t1", 1, 4), Task("t2", 2, 6), Task("t3", 2, 12)]; B = A + [Task("t4", 1, 8)]
H = hyperperiod(B)
rm, edf = simulate(B, H, "RM"), simulate(B, H, "EDF")
R["hyper"] = H; R["rm_misses"] = rm["misses"]; R["edf_misses"] = edf["misses"]; R["rm_tl"] = rm["timeline"];
R["edf_tl"] = edf["timeline"]
R["diff_ticks"] = [t for t in range(H) if rm["timeline"][t] != edf["timeline"][t]]
o = priority_order(B); R["rta"] = [(k.name, *rta(o, i)) for i, k in enumerate(o)]
R["u3"], R["b3"], R["u4"], R["b4"] = total_utilization(A), rm_bound(3), total_utilization(B), rm_bound(4)
# stacked RM / EDF chart + the four required PNGs
fig, ax = plt.subplots(2, 1, figsize=(7.2, 4.6), sharex=True)
draw_schedule(ax[0], B, rm, "Rate Monotonic (hyperperiod 24)", H); draw_schedule(ax[1], B, edf, "Earliest
Deadline First", H); ax[1].set_xlabel("time (ticks)")
plt.tight_layout(); plt.savefig("figs/rm_edf.png", dpi=160); plt.close()
for nm, res in (("rm", rm), ("edf", edf)):
    fig, ax = plt.subplots(figsize=(7.2, 2.4)); draw_schedule(ax, B, res, nm.upper(), H);
ax.set_xlabel("time (ticks)"); plt.tight_layout(); plt.savefig(f"figs/gantt_{nm}.png", dpi=160); plt.close()
# priority inversion
none = simulate_inversion(INV_JOBS, 16, pip=False); pip = simulate_inversion(INV_JOBS, 16, pip=True)
R["inv"] = {k: dict(finish=v["finish"], wait=v["waiting"], tl=[n for n, _ in v["timeline"]]) for k, v in
(("none", none), ("pip", pip))}
fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.6), sharey=True)
draw_inversion(ax[0], none, "No protocol (unbounded inversion)", 16); draw_inversion(ax[1], pip, "Priority
Inheritance Protocol", 16)
plt.tight_layout(); plt.savefig("figs/inversion.png", dpi=160); plt.close()
for nm, res in (("none", none), ("pip", pip)):
    fig, ax = plt.subplots(figsize=(5.2, 2.4)); draw_inversion(ax, res, "no protocol" if nm == "none" else
"PIP", 16); plt.tight_layout(); plt.savefig(f"figs/inversion_{nm}.png", dpi=160); plt.close()
# Lab 1: DM vs RM
imp = [Task("a", 2, 6), Task("b", 2, 8)]; con = [Task("a", 2, 6), Task("b", 2, 8, 3)]
def cmp(ts):
    h = hyperperiod(ts); r, d = simulate(ts, h, "RM"), simulate(ts, h, "DM")
    return dict(same=r["timeline"] == d["timeline"], rm_miss=r["misses"], dm_miss=d["misses"],
rm=r["timeline"], dm=d["timeline"])
R["dm_implicit"], R["dm_constr"] = cmp(imp), cmp(con)
ts = con; h = hyperperiod(ts)
fig, ax = plt.subplots(2, 1, figsize=(6.4, 3.4), sharex=True)
draw_schedule(ax[0], ts, simulate(ts, h, "RM"), "RM priorities (D ignored): task b misses D = 3", h);
draw_schedule(ax[1], ts, simulate(ts, h, "DM"), "Deadline Monotonic: all deadlines met", h)
plt.tight_layout(); plt.savefig("figs/dm.png", dpi=160); plt.close()
# Lab 2: bound sweep
ns = list(range(1, 11)); bs = [rm_bound(n) for n in ns]; R["bounds"] = bs
fig, ax = plt.subplots(figsize=(5.6, 2.6)); ax.plot(ns, bs, "o-", label="RM bound n(2^(1/n)-1)");
ax.axhline(1, color="g", ls="--", label="EDF bound = 1.0"); ax.axhline(math.log(2), color="r", ls=":",
label="ln 2 = 0.693")
ax.set_xlabel("number of tasks n"); ax.set_ylabel("utilization bound"); ax.set_ylim(.6, 1.05); ax.legend();
ax.grid(alpha=.3); plt.tight_layout(); plt.savefig("figs/bounds.png", dpi=160); plt.close()
# Lab 3: LLF
R["switches"] = {p: simulate(B, H, p)["switches"] for p in ("RM", "EDF", "LLF")}
R["llf_misses"] = simulate(B, H, "LLF")["misses"]
llf_sets = [("4-task set", B, H)]
ts2 = [Task("x", 3, 8), Task("y", 3, 8), Task("z", 2, 8)]; R["switches2"] = {p: simulate(ts2, 8,
p)["switches"] for p in ("RM", "EDF", "LLF")}
fig, ax = plt.subplots(figsize=(7.2, 2.4)); draw_schedule(ax, B, simulate(B, H, "LLF"), "LLF over the
hyperperiod", H); ax.set_xlabel("time (ticks)"); plt.tight_layout(); plt.savefig("figs/llf.png", dpi=160);
plt.close()
# Lab 4: second medium task
jobs4 = INV_JOBS + [dict(name="M2", prio=2, arrive=4, C=3, res=False)]
n4, p4 = simulate_inversion(jobs4, 20, False), simulate_inversion(jobs4, 20, True)
```

```
R["lab4"] = dict(none=n4["waiting"]["H"], pip=p4["waiting"]["H"], fin_none=n4["finish"]["H"],
fin_pip=p4["finish"]["H"])
json.dump(R, open("results/mp21.json", "w"))
print(R["inv"]["none"]["finish"], R["inv"]["none"]["wait"], R["inv"]["pip"]["finish"],
R["inv"]["pip"]["wait"]); print(R["lab4"]); print(R["switches"], R["switches2"], R["llf_misses"])
print(R["dm_implicit"]["same"], R["dm_constr"]["same"], R["dm_constr"]["rm_miss"],
R["dm_constr"]["dm_miss"]); print(R["rm_misses"], R["edf_misses"], R["diff_ticks"]);
print(R["inv"]["none"]["tl"], R["inv"]["pip"]["tl"])
```

B. Mini Project 2.2

sched_tool.py

```
"""Mini Project 2.2: RM vs EDF schedulability comparison tool.
Usage:  python sched_tool.py tasks.csv [chart.png]     (CSV columns: name,C,T[,D])      python sched_tool.py
--demo"""
import csv, sys, math
from rts_sim import *

def load_csv(path):
    tasks = []
    with open(path, newline="") as f:
        for r in csv.DictReader(f):
            d = (r.get("D") or "").strip()
            tasks.append(Task(r["name"].strip(), int(r["C"]), int(r["T"]), int(d) if d else None))
    return tasks

def validate(tasks):
    if not tasks: return "empty task set"
    for t in tasks:
        if t.C <= 0 or t.T <= 0 or t.D <= 0: return f"{t.name}: C, T and D must be positive"
        if t.D > t.T: return f"{t.name}: D > T (arbitrary deadlines) is not supported"
        if t.C > t.D: return f"{t.name}: C > D, the task can never meet its deadline"
    return None

def analyze(tasks, cap=500):
    err = validate(tasks)
    if err: return dict(error=err)
    constrained = any(t.D < t.T for t in tasks)
    n, U = len(tasks), total_utilization(tasks)
    order = priority_order(tasks, "DM" if constrained else "RM")      # DM ordering whenever some D < T
    out = dict(n=n, U=U, bound=rm_bound(n), constrained=constrained, order=[t.name for t in order],
               per_task=[(t.name, t.C, t.T, t.D, t.U) for t in tasks], infeasible=U > 1 + 1e-12)
    out["bound_pass"] = (not constrained) and U <= out["bound"] + 1e-12
    out["edf_pass"] = U <= 1 + 1e-12                                   # exact for D = T (Section 2.3.1)
    out["rta"] = None
    if not out["infeasible"] and not out["bound_pass"]:                # fall back to exact analysis
(Section 2.5.2)
        rows = []
        for i, t in enumerate(order): R, it, ok = rta(order, i); rows.append((t.name, R, it, t.D, ok))
        out["rta"] = rows; out["rm_ok"] = all(r[4] for r in rows)
    else: out["rm_ok"] = out["bound_pass"]
    if out["infeasible"]: out["rm_ok"] = False
    H = hyperperiod(tasks); out["hyper"] = H; out["horizon"] = min(H, cap)
    if not out["infeasible"]:
        out["rm_sim_misses"] = simulate(tasks, out["horizon"], "DM" if constrained else "RM")["misses"]
        out["edf_sim_misses"] = simulate(tasks, out["horizon"], "EDF")["misses"]
    return out

def report(name, tasks):
    a = analyze(tasks); L = [f"=== {name} ==="]
    if "error" in a: return "\n".join(L + [f"INPUT ERROR: {a['error']}"])
    for nm, C, T, D, u in a["per_task"]: L.append(f"  {nm}: C={C} T={T} D={D} u={u:.3f}")
    L.append(f"  n={a['n']}  total U={a['U']:.3f}  hyperperiod={a['hyper']}")
    if a["infeasible"]:
        L.append("  VERDICT: INFEASIBLE (U > 1): no scheduler can meet all deadlines on one processor.")
        return "\n".join(L)
    if a["constrained"]: L.append(f"  Constrained deadlines present: Deadline Monotonic order {a['order']};
Liu-Layland bound not applicable, using RTA")
    else: L.append(f"  Liu-Layland bound n(2^(1/n)-1) = {a['bound']:.3f}: {'PASS' if a['bound_pass'] else
'FAIL (inconclusive)'}")
    if a["rta"]:
        for nm, R, it, D, ok in a["rta"]: L.append(f"    RTA {nm}: R={R} (iterations {it}) vs D={D}: {'OK'
if ok else 'MISS'}")
    L.append(f"  RM/DM verdict: {'SCHEDULABLE' if a['rm_ok'] else 'NOT schedulable'}")
    L.append(f"  EDF (U <= 1): {'PASS' if a['edf_pass'] else 'FAIL'}" + (" (necessary only when D < T)" if
a["constrained"] else "")
             + f"; simulated deadline misses over {a['horizon']} ticks: RM/DM={len(a['rm_sim_misses'])},
EDF={len(a['edf_sim_misses'])}")
    return "\n".join(L)
```

```
def compare_chart(tasks, path):
    import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
    a = analyze(tasks)
    if "error" in a or a["infeasible"]: return False
    H = a["horizon"]; pol = "DM" if a["constrained"] else "RM"
    fig, ax = plt.subplots(2, 1, figsize=(7.2, 1.2 + 0.55 * len(tasks) * 2), sharex=True)
    draw_schedule(ax[0], tasks, simulate(tasks, H, pol), f"{pol} (hyperperiod {a['hyper']})", H);
draw_schedule(ax[1], tasks, simulate(tasks, H, "EDF"), "EDF", H)
    ax[1].set_xlabel("time (ticks)"); plt.tight_layout(); plt.savefig(path, dpi=150); plt.close(); return
True


SETS = {
 "S1 Section 2.2.2 (3 tasks)": [Task("t1", 1, 4), Task("t2", 2, 6), Task("t3", 2, 12)],
 "S2 Section 2.3.2 (4 tasks)": [Task("t1", 1, 4), Task("t2", 2, 6), Task("t3", 2, 12), Task("t4", 1, 8)],
 "S3 overloaded (U > 1)": [Task("a", 3, 5), Task("b", 4, 6), Task("c", 1, 4)],
 "S4 constrained deadline": [Task("a", 2, 6), Task("b", 2, 8, 3), Task("c", 1, 12)],
 "S5 eight tasks, U = 0.70": [Task(f"t{i+1}", c, p) for i, (c, p) in enumerate([(1, 10), (2, 20), (2, 20),
(3, 40), (3, 40), (4, 40), (6, 80), (6, 80)])],
 "S6 eight harmonic tasks, U = 0.80": [Task(f"t{i+1}", c, p) for i, (c, p) in enumerate([(1, 10), (2, 20),
(2, 20), (4, 40), (4, 40), (4, 40), (8, 80), (8, 80)])],
}

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] != "--demo":
        ts = load_csv(sys.argv[1]); print(report(sys.argv[1], ts))
        if len(sys.argv) > 2: print("chart written" if compare_chart(ts, sys.argv[2]) else "no chart
(invalid or infeasible set)")
    else:
        for k, v in SETS.items(): print(report(k, v)); print()
```

exp22.py

```
import json
from sched_tool import *
res = {}
for k, ts in SETS.items():
    a = analyze(ts); res[k] = {x: v for x, v in a.items() if x not in ("per_task",)}
    tag = k.split()[0].lower()
    if tag in ("s1", "s2", "s4", "s6"): compare_chart(ts, f"figs/cmp_{tag}.png")
json.dump(res, open("results/mp22.json", "w"), default=str); print("ok")
```

C. Mini Project 2.3

partition.py

```
"""Mini Project 2.3: partitioned multiprocessor scheduling with bin-packing heuristics (Section 2.8)."""
from rts_sim import Task, rm_bound, total_utilization


def _slack(core, t):
    """Remaining capacity of a core if t were added, measured against rm_bound for the NEW task count."""
    n = len(core) + 1
    return rm_bound(n) - (sum(x.U for x in core) + t.U)


def _admissible(core, t): return _slack(core, t) >= -1e-12          # load + u <= rm_bound(n + 1)


def _pack(tasks, m, choose):
    cores = [[] for _ in range(m)]; unplaced = []
    for t in tasks:
        ok = [i for i in range(m) if _admissible(cores[i], t)]
        if not ok: unplaced.append(t); continue
        cores[choose(ok, cores, t)].append(t)
    return dict(cores=cores, unplaced=unplaced)


def first_fit(tasks, m):             return _pack(tasks, m, lambda ok, c, t: ok[0])
def first_fit_decreasing(tasks, m):  return _pack(sorted(tasks, key=lambda t: -t.U), m, lambda ok, c, t:
ok[0])
def best_fit_decreasing(tasks, m):   return _pack(sorted(tasks, key=lambda t: -t.U), m, lambda ok, c, t:
min(ok, key=lambda i: _slack(c[i], t)))
def worst_fit(tasks, m):             return _pack(tasks, m, lambda ok, c, t: max(ok, key=lambda i:
_slack(c[i], t)))
HEURISTICS = {"FF": first_fit, "FFD": first_fit_decreasing, "BFD": best_fit_decreasing, "WF": worst_fit}


def summarize(res, m):
    loads = [sum(t.U for t in c) for c in res["cores"]]
    return dict(success=not res["unplaced"], cores_used=sum(1 for c in res["cores"] if c), loads=loads,
                placed_U=sum(loads), pct_of_m=100 * sum(loads) / m, unplaced=[t.name for t in
res["unplaced"]])
```

```
def mk(prefix, specs): return [Task(f"{prefix}{i+1}", c, p) for i, (c, p) in enumerate(specs)]
```

exp23.py

```
import json, math, matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt, numpy as np
from partition import *
from rts_sim import simulate, hyperperiod, draw_schedule
plt.rcParams.update({"font.size": 8})
M = 4
SETS = {
 "S1 balanced (16 small)": mk("b", [(6, 40), (7, 40)] * 8),
 "S2 skewed (10 small + 3 big, small first)": mk("s", [(2, 20)] * 10 + [(12, 20)] * 3),
 "S3 tight (16 x u=0.18)": mk("t", [(9, 50)] * 16),
 "S4 adversarial (5 x u=0.51)": mk("a", [(51, 100)] * 5),
 "S5 order-sensitive (10 mixed)": mk("o", [(4, 40), (7, 10), (7, 40), (2, 40), (8, 20), (3, 10), (4, 20),
(4, 10), (3, 10), (7, 20)]),
}
out = {}
for sname, ts in SETS.items():
    out[sname] = dict(U=total_utilization(ts), n=len(ts), lower=math.ceil(total_utilization(ts)), res={})
    for h, f in HEURISTICS.items():
        r = f(ts, M); s = summarize(r, M); out[sname]["res"][h] = s
        print(f"{sname:45s} {h:3s} ok={s['success']!s:5} cores={s['cores_used']} loads={[round(x,3) for x in
s['loads']]} U%m={s['pct_of_m']:.1f} unplaced={s['unplaced']}")
json.dump(out, open("results/mp23.json", "w"))
# grouped bar: achieved utilization (% of m)
fig, ax = plt.subplots(figsize=(7.2, 3)); names = list(SETS); w = .2
for j, h in enumerate(HEURISTICS):
    vals = [out[n]["res"][h]["pct_of_m"] for n in names]
    bars = ax.bar(np.arange(len(names)) + (j - 1.5) * w, vals, w, label=h)
    for b, n in zip(bars, names):
        if not out[n]["res"][h]["success"]: ax.text(b.get_x() + w / 2, b.get_height() + 1, "x", ha="center",
color="red", fontsize=9, fontweight="bold")
ax.set_xticks(range(len(names)), [n.split(" ")[0] for n in names]); ax.set_ylabel("placed utilization (% of
m = 4)"); ax.set_title("Placed utilization per heuristic (red x = some tasks could not be placed)");
ax.legend(ncol=4); ax.grid(axis="y", alpha=.3)
plt.tight_layout(); plt.savefig("figs/heur.png", dpi=160); plt.close()
# adversarial: per-core loads (FFD) + bound lines
adv = out["S4 adversarial (5 x u=0.51)"]["res"]["FFD"]
fig, ax = plt.subplots(figsize=(5.6, 2.6)); ax.bar([f"core {i+1}" for i in range(M)], adv["loads"],
color="#2e75b6"); ax.axhline(rm_bound(1), color="g", ls="--", label="bound for 1 task = 1.0");
ax.axhline(rm_bound(2), color="r", ls=":", label="bound for 2 tasks = 0.828")
ax.set_ylabel("core utilization"); ax.set_title("S4: one 0.51 task per core; the fifth task fits nowhere");
ax.legend(loc="lower right"); ax.set_ylim(0, 1.15); plt.tight_layout(); plt.savefig("figs/adv.png",
dpi=160); plt.close()
# per-core Gantt for S5 under FFD
ts = SETS["S5 order-sensitive (10 mixed)"]; r = first_fit_decreasing(ts, M); H = hyperperiod(ts); misses =
{}
fig, axs = plt.subplots(M, 1, figsize=(7.2, 6.2), sharex=True, gridspec_kw=dict(height_ratios=[max(1,
len(c)) + .6 for c in r["cores"]]))
for i, c in enumerate(r["cores"]):
    res = simulate(c, H, "RM"); misses[f"core {i+1}"] = len(res["misses"])
    draw_schedule(axs[i], c, res, f"Core {i+1}: {len(c)} tasks, U = {sum(t.U for t in c):.3f} (RM bound
{rm_bound(len(c)):.3f}), deadline misses = {len(res['misses'])}", H)
axs[-1].set_xlabel("time (ticks)"); plt.tight_layout(); plt.savefig("figs/percore.png", dpi=160);
plt.close()
json.dump(dict(misses=misses, cores=[[(t.name, t.C, t.T) for t in c] for c in r["cores"]], H=H),
open("results/mp23_gantt.json", "w")); print(misses)
```